RATING PLANETTERP REVIEWS WITH TRANSFORMER MODELS

Mouli Banga

CMSC320 - 12/11/2025

CONTEXT

PlanetTerp is a community for students to leave honest reviews on their professors for courses that they've enrolled in.

PlanetTerp includes...

24,867 courses

14,035 professors

42,688 reviews

323,979 course grades

Maksym Morawski
CMSC320
★★★★★
Expecting an A
Anonymous
05/09/2025

Fantastic, he does a great job and has an easy enough course load. Projects are fun but the overall number of them is unpredictable but he's great!

THE GOAL

Do students' reviews accurately reflect the rating they leave for their professors?

By using machine learning algorithms, we can see if the reviews written for each professor corresponds with the score they end up giving.

DATASET

SOURCE

All professor data and associated reviews were collected using the **PlanetTerp API.**

PROFESSORS

- Abani Pradhan
- Abigail McEwen
- Abolhassan Jawahery
- Abram Kagan
- Adam Hsieh

STATISTICS

Total Reviews: 40
Professors: 5
Columns: professor, course, review, rating, expected_grade, created
Train / validation split: 32 / 8

NOTES

- Only kept reviews with non-empty text and a valid 1–5 star rating.
- Only included professors with ≥ 5 usable reviews.
- Saved the final dataset for reproducibility.

THE PLAN

- Collect reviews from PlanetTerp.
- Build, clean, and label the dataset.
- Turn reviews into model inputs.
- Fine-tune the transformer in PyTorch.
- Evaluate and show predictions.

ML MODEL

Model: DistilBERT (base-uncased):

- Pre-trained transformer language model from Hugging Face
- Output layer has 5 classes (for star ratings 1–5, mapped to labels 0–4)
- Fine-tuned only on PlanetTerp review text (no extra features)

Training Configuration:

- Framework: PyTorch + Hugging Face transformers
- Optimizer: AdamW, learning rate 2e-5
- Epochs: 3 (small dataset)
- Data split: 80% train / 20% validation

RESULT

```
===== Epoch 1/3 =====
Train loss: 1.6016
Val   loss: 1.5478 | Val accuracy: 0.3750

===== Epoch 2/3 =====
Train loss: 1.5271
Val   loss: 1.5041 | Val accuracy: 0.3750

===== Epoch 3/3 =====
Train loss: 1.4420
Val   loss: 1.4941 | Val accuracy: 0.3750
```

Validation Metrics

- Accuracy: ~37.5%
- Validation loss: ≈ 1.49
- Random guess over 5 classes ≈ 20% accuracy → model is better than random

ANALYSIS

What the model seems to learn:

- It does pick up overall sentiment:
  - Very positive reviews often predicted as 4–5 stars
  - More negative reviews sometimes still predicted as 4 stars
  - Clear behavior: the model leans toward predicting 4 stars a lot.
- Why performance is limited:
  - Very small dataset: only 40 total reviews.
  - Likely class imbalance (many 4–5 star ratings, fewer low scores).
  - Only trained for 3 epochs on CPU, so fine-tuning is shallow.

```
---
Review: The most "reasonab
True rating:      4
Predicted rating: 4

---
Review: Phys 405 is diffic
True rating:      4
Predicted rating: 4

---
Review: don't take this cl
True rating:      2
Predicted rating: 4

---
Review: Not a great profes
True rating:      2
Predicted rating: 4

---
Review: I love Dr. Pradhan
True rating:      5
Predicted rating: 4
```

SUMMARY

What I Built:

- Collected PlanetTerp reviews for 5 UMD professors using the API.
- Created a small labeled dataset of 40 reviews (text + 1–5 star rating).
- Fine-tuned a DistilBERT transformer in PyTorch as a 5-class classifier.

How well it worked:

- Achieved ~37.5% validation accuracy, clearly above random guessing.
- Model captures positive vs negative tone, but struggles to hit the exact star rating with so little data.

If I had more time / data:

- Pull reviews for many more professors to grow the dataset.
- Balance the star ratings and maybe try a few different architectures or hyperparameters.

THANK YOU!

```
!pip install -q requests pandas torch transformers scikit-learn

import requests
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from transformers import AutoTokenizer, AutoModelForSequenceClassification


"""
function that will return: list of professor names + list of all their reviews
params : target - # of profs we want (5), min_reviews - each prof must have at least this many usable reviews, limit -
"""

def get_professors_with_reviews(target=5, min_reviews=5, limit=100):
    offset = 0          # where in the list of professors to start
    selected_profs = [] # empty list where we'll store the names of the 5 professors
    review_rows = []    # empty list where we'll store all the reviews (each review = one row)

    # keep looping until we haven't found enough profs
    while len(selected_profs) < target:
        url = f"https://planetterp.com/api/v1/professors?limit={limit}&offset={offset}"
        response = requests.get(url)

        # if the request failed, stop
        if response.status_code != 200:
            print("Failed to fetch professors:", response.status_code)
            break

        # parse the JSON into Python objects
        batch = response.json()

        # if the batch is empty, there are no more professors
        if not batch:
            print("No more professors returned from API.")
            break

        # go through each professor in this batch
        for prof in batch:
            # if we already have enough professors, stop checking this batch
            if len(selected_profs) >= target:
                break

            name = prof["name"]  # professor's name as a string

            # fetch this specific professor with their reviews
            prof_url = f"https://planetterp.com/api/v1/professor?name={name}&reviews=true"
            r = requests.get(prof_url)
            if r.status_code != 200:
                # if this request fails, skip this prof
                continue

            prof_data = r.json()
            reviews = prof_data.get("reviews", [])

            # we only keep reviews that have BOTH text and a rating
            usable_reviews = []
            for review in reviews:
                text = review.get("review", "")
                rating = review.get("rating", None)

                # skip if text is empty or rating is missing
                if not text or rating is None:
                    continue

                usable_reviews.append(review)

            # if this prof doesn't have enough usable reviews, skip them
            if len(usable_reviews) < min_reviews:
                continue

            # at this point, we accept this professor
            selected_profs.append(name)

            # add each of their usable reviews as a row in our dataset
```

```
            # add each of their usable reviews as a row in our dataset
            for review in usable_reviews:
                review_rows.append({
                    "professor": name,
                    "course": review.get("course", None),
                    "review": review.get("review", ""),
                    "rating": review.get("rating", None),
                    "expected_grade": review.get("expected_grade", None),
                    "created": review.get("created", None),
                })

        # move to the next "page" of professors for the next loop
        offset += limit

    return selected_profs, review_rows


# Actually run the collection
TARGET_PROFS = 5
MIN_REVIEWS_PER_PROF = 5

selected_profs, review_rows = get_professors_with_reviews(
    target=TARGET_PROFS,
    min_reviews=MIN_REVIEWS_PER_PROF
)

print("Selected professors:")
for name in selected_profs:
    print(" -", name)

print("\nTotal reviews collected:", len(review_rows))

# Turn the list of review dicts into a DataFrame
df = pd.DataFrame(review_rows)

# drop rows with missing rating or empty review text
df = df[
    df["rating"].notna()
    & df["review"].notna()
    & (df["review"].str.strip() != "")
]

print("\nFinal dataset shape:", df.shape)
print(df.head())

# save dataset so it's clear how it was prepared
df.to_csv("professor_review_dataset.csv", index=False)
print("\nSaved dataset to 'professor_review_dataset.csv'")


"""
prepare data for modeling - review text and rating
"""

# we only need the review text and rating for modeling
data = df[["review", "rating"]].copy()
data["rating"] = data["rating"].astype(int)

# keep only ratings between 1 and 5 (should already be, but just in case)
data = data[(data["rating"] >= 1) & (data["rating"] <= 5)]

# map ratings 1-5 -> labels 0-4 (this is what the model predicts)
data["label"] = data["rating"] - 1

# split into train and validation sets (80% train, 20% val)
train_df, val_df = train_test_split(
    data,
    test_size=0.2,
    random_state=42
)

print("\nTrain size:", len(train_df), "Val size:", len(val_df))

"""
build dataset + dataloader - pytorch. adapted from common hugging face examples
"""

model_name = "distilbert-base-uncased"
```

```
# This loads DistilBERT's tokenizer from Hugging Face.
# Pattern similar to Hugging Face tutorials for sequence classification.
tokenizer = AutoTokenizer.from_pretrained(model_name)

class ReviewDataset(Dataset):
    def __init__(self, df, tokenizer, max_length=256):
        self.df = df.reset_index(drop=True)
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        # number of rows in the DataFrame
        return len(self.df)

    def __getitem__(self, idx):
        # get the text and label for this index
        text = self.df.loc[idx, "review"]
        label = int(self.df.loc[idx, "label"])

        # tokenize the text
        # using DistilBERT tokenizer
        encoded = self.tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=self.max_length,
            return_tensors="pt"
        )

        return {
            "input_ids": encoded["input_ids"].squeeze(0),
            "attention_mask": encoded["attention_mask"].squeeze(0),
            "labels": torch.tensor(label, dtype=torch.long),
        }

# create Dataset objects for train and validation data
train_dataset = ReviewDataset(train_df, tokenizer)
val_dataset = ReviewDataset(val_df, tokenizer)

# DataLoaders create batches and shuffle the training data
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=4, shuffle=False)

"""
define model + simple training loop - adapted from pytorch examples online
"""

device = torch.device("cpu")
print("Using device:", device)

num_labels = 5  # 5 possible star ratings: 1, 2, 3, 4, 5

#follows the standard Hugging Face text classification pattern.
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=num_labels
).to(device)

# AdamW optimizer is commonly used with transformers (taken from online examples/docs)
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)


def train_one_epoch(model, data_loader, optimizer, device):
    """
    Train for one epoch:
      - loop over all batches
      - forward pass
      - backward pass
      - optimizer step

    """
    model.train()
    total_loss = 0.0

    for batch in data_loader:
        optimizer.zero_grad()
```

```
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        loss = outputs.loss

        loss.backward()

        optimizer.step()

        total_loss += loss.item()

    # return average loss over all batches
    return total_loss / len(data_loader)


def evaluate(model, data_loader, device):
    """
    Evaluation loop:
      - no gradient updates
      - compute loss and accuracy on the validation set

    """
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for batch in data_loader:
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels = batch["labels"].to(device)

            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )
            loss = outputs.loss
            logits = outputs.logits

            total_loss += loss.item()

            preds = torch.argmax(logits, dim=-1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

    avg_loss = total_loss / len(data_loader) if len(data_loader) > 0 else 0.0
    accuracy = correct / total if total > 0 else 0.0
    return avg_loss, accuracy

"""
train
"""

EPOCHS = 3  # small number of epochs because dataset is small

for epoch in range(EPOCHS):
    print(f"\n===== Epoch {epoch + 1}/{EPOCHS} =====")
    train_loss = train_one_epoch(model, train_loader, optimizer, device)
    val_loss, val_acc = evaluate(model, val_loader, device)

    print(f"Train loss: {train_loss:.4f}")
    print(f"Val   loss: {val_loss:.4f} | Val accuracy: {val_acc:.4f}")


"""
predictions
"""

def predict_rating(text, model, tokenizer, device):
```

```
    """
    Given a review text, return the model's predicted star rating (1–5).
    This uses the same tokenization pattern as above.
    """
    model.eval()
    with torch.no_grad():
        encoded = tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=256,
            return_tensors="pt"
        )
        encoded = {k: v.to(device) for k, v in encoded.items()}
        outputs = model(**encoded)
        logits = outputs.logits
        pred_label = int(torch.argmax(logits, dim=-1).item())
        # convert label 0–4 back to rating 1–5
        return pred_label + 1

print("\nSome validation examples (true vs predicted):")
for i in range(min(5, len(val_df))):
    row = val_df.iloc[i]
    text = row["review"]
    true_rating = int(row["rating"])
    pred_rating = predict_rating(text, model, tokenizer, device)

    print("\n---")
    print("Review:", text[:200].replace("\n", " "), "...")
    print("True rating:     ", true_rating)
    print("Predicted rating:", pred_rating)
```

```
Selected professors:
 - Abani Pradhan
 - Abigail McEwen
 - Abolhassan Jawahery
 - Abram Kagan
 - Adam Hsieh

Total reviews collected: 40

Final dataset shape: (40, 6)
       professor   course                                            review  \
0  Abani Pradhan     None  Pretty funny guy and he knows his stuff too. P...
1  Abani Pradhan  NFSC112  Easy class as long as you actually watch the l...
2  Abani Pradhan  NFSC112  Didn't learn anything, but didn't need to. Eas...
3  Abani Pradhan  NFSC112  This class had a lot of overlap with NFSC100 b...
4  Abani Pradhan  NFSC112  I love Dr. Pradhan, he makes lectures pretty i...

   rating expected_grade                      created
0       5             B+  2022-12-11T04:18:01.840304Z
1       2             B-  2023-12-06T04:55:26.042809Z
2       4              A  2023-12-23T18:57:10.860075Z
3       5              A  2025-10-29T02:57:48.589036Z
4       5              A  2025-11-14T16:08:40.411298Z

Saved dataset to 'professor_review_dataset.csv'

Train size: 32 Val size: 8
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/token
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%                    48.0/48.0 [00:00<00:00, 861B/s]

config.json: 100%                    483/483 [00:00<00:00, 8.59kB/s]

vocab.txt: 100%                    232k/232k [00:00<00:00, 2.06MB/s]

tokenizer.json: 100%                    466k/466k [00:00<00:00, 5.10MB/s]

Using device: cpu
model.safetensors: 100%                    268M/268M [00:02<00:00, 179MB/s]
Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

===== Epoch 1/3 =====
Train loss: 1.6016
Val   loss: 1.5478 | Val accuracy: 0.3750

===== Epoch 2/3 =====
Train loss: 1.5271
Val   loss: 1.5041 | Val accuracy: 0.3750

===== Epoch 3/3 =====
Train loss: 1.4420
Val   loss: 1.4941 | Val accuracy: 0.3750

Some validation examples (true vs predicted):

---
```